DevDesk 智能中枢
技术调研报告

——从开发者桌面工具集到智能中枢的战略技术评估

版本：v2.0

日期：2026-10-21

编制：DevDesk 项目组

分类：内部技术报告

摘 要

本报告是 DevDesk 从"开发者桌面工具集"向"智能中枢"战略转型的技术调研总成。报告基于七路调研，系统评估了智能中枢各关键技术方向的成熟度、可用方案与本项目适配性。

核心结论：DevDesk 的智能中枢路线图（A 至 E 五期）方向正确，不存在"全期报废"的致命风险。成熟 Harness（智能体运行时，即触发调度、工具注册、权限闸、审计、预算护栏等围绕智能体循环的宿主机制统称）骨架在开源社区已有充分验证；PRD 到 MR 流水线与行业共识一致——MR 留人审是当前所有主流产品的共同选择。最大风险在协议层：C 期如不做 MCP 适配缝，将在 2027-2028 年标准化浪潮中被迫返工。A 期即可开始埋协议适配缝（约一天工作量），成本远低于事后重构。

报告共六章：调研背景、现状分析、运行原理、调研结论、路线图结合、结论与建议。附三份附录：引用清单、决策记录卡、术语表。

关键词：智能中枢；AI Agent；Harness 运行时；AgentLoop；MCP 协议；技能（Skill）；PRD-to-MR 流水线；C++20

目 次

摘要

第一章 调研背景与目标

1.1 智能中枢定位

1.2 七路调研综述

1.3 调研方法与限制

第二章 智能体现状技术分析

2.1 四层架构

2.2 执行循环机制

2.3 触发调度体系

2.4 护栏与审计

2.5 工具与扩展点

2.6 模块依赖反转

第三章 运行原理

3.1 Harness 基本原理与工作流程

3.2 技能的调用与实现

3.3 MCP 的调用与实现

3.4 用户如何扩展能力

第四章 调研结论

4.1 行业格局

4.2 成熟 Harness 机制

4.3 工具生态与技能

4.4 GitLab 对接

4.5 可靠性与自主性

4.6 未来趋势与押注

第五章 与本项目计划的结合

5.1 A 期：收口

5.2 B 期：对接能力

5.3 C 期：扩展机制

5.4 D 期：自主性

5.5 E 期：PRD→MR 流水线

5.6 已拍板决策

5.7 设计原则

5.8 吸收映射

5.9 结论→计划对照

第六章 结论与建议

6.1 总判断

6.2 最大风险与对冲

6.3 三条近期建议

附录 A 引用清单

附录 B 决策记录卡

附录 C 术语表

第 一章 调研背景与目标

> **本章要点：**界定 DevDesk 智能中枢的战略定位（特化三轴、轻资产原则），综述七路调研的范围与方法，说明调研的网络受限环境与引用口径。

1.1 智能中枢定位

DevDesk 是面向开发者的跨平台桌面工具集（C++20 GUI 应用），现有七个功能页面。2026 年 10 月，项目确定战略方向为**智能中枢**：形态借鉴 Claude Code 和 OpenCode 已验证的"智能体 + Harness"骨架。这里的 Harness 指围绕智能体循环的一整套宿主机制——触发调度、工具注册、权限闸、预算护栏、运行审计、任务记忆——它们不是模型的一部分，而是为模型提供"可安全执行任务的运行环境"的工程系统。Harness 的概念贯穿本报告全文，具体工作原理见第三章。

智能中枢特化在三条轴上：**（1）无人值守**——触发驱动（定时 / 事件 / 自唤醒）+ 失败重试 + 验收自评 + 失败升级通知。人不在环，可靠性优先于交互性。**（2）C++ 原生桌面常驻**——系统托盘、可视化审计、桌面感知（剪贴板 / 窗口 / 文件状态），这是 Web 或 Electron 方案做不到的差异化能力。**（3）业务绑定**——自有模块工具面 + 公司内部 GitLab / 敏捷平台对接，外部产品不会为这些场景做深度定制。

路线方针为"走前人铺好的路，继而发展出自己的路"：每个设计取舍先问"前人（Claude Code / OpenCode）有没有已验证的答案"，有就照搬机制模式；没有答案、或答案不落在三条特化轴上，才自研。

轻资产原则（ADR 0004）：核心只保留"机制与接口"——智能体循环、工具注册、事件总线、触发调度、权限闸、审计、预算护栏。一切具体业务能力（文件工具、GitLab、搜索、技能包、角色定义）一律以可插拔形态接入，不进核心。接口质量的硬判据为新能力以"注册 / 放文件"形态接入的步骤数——工具 4 步、事件 2 步即标准线。协议层面同样贯彻可替换原则：模型端点与工具协议都是可替换层（core::LlmEndpoint / ToolRegistry 即为接口缝合处）。

1.2 七路调研综述

为支撑路线图各期决策，项目于 2026-10-21 集中开展了七路外部调研：

表 1-1 七路调研总览

#	调研	主要内容	产出文档
1	行业趋势	21 次抓取，覆盖 Claude Code / Copilot / Cursor / n8n / Raycast / OpenHands 等 7 个主题	06
2	OpenCode 源码吸收	对 opencode v2.1.1 的机制拆解：权限模型、Agent 角色体系、工具定义 Schema、会话与持久化	07
3	GitHub 生态深扫	24 次抓取，6 个开源 Agent 框架盘点、5 个技能候选、5 个克隆仓库推荐	08
4	GitLab 工具链	17 次抓取，reviewdog / Danger 审查方案、GitLab MCP Server 现状、webhook 事件接入	09
5	学术文献	14 次有效抓取，17 篇论文，总结验收自评、失败重试、自唤醒、多角色交付四类设计建议	10
6	未来趋势	16 次抓取，2027-2028 推演：模型能力 / 协议标准化 / 开发工作流 / 桌面 Agent 四轴	11
7	AI Agent 手册	阿里云 AI Agent Handbook 23/30 章精读，11 条直接吸收 + 9 条已有等价	12

1.3 调研方法与限制

所有调研在公司网络环境下完成，使用系统 Edge 无头模式抓取目标页面正文。受限于公司网关屏蔽和部分站点反爬机制，约 20-30% 抓取尝试失败（Bing 搜索被全量屏蔽、知乎反爬 403、部分站点 SPA 渲染不全）。关键结论均经多源交叉验证，单一来源的个别断言已标注。所有引用均附抓取日期（2026-10-21）和完整 URL，见附录 A。

第 二章 智能体现状技术分析

> **本章要点：**以 DevDesk 现有代码库为基准，系统展示智能体模块的四层架构、AgentLoop 执行循环、六种触发调度、预算护栏与三表审计、21 个工具清单、以及模块依赖反转设计。

2.1 四层架构

DevDesk 采用四层分层架构：应用入口层（main）→ 模块层（module_*，7 个 Router 页面）→ 核心层（core：Result、数据库、路由、主题、工具注册、事件总线、LLM 端点）→ 平台层（platform：HTTP 客户端/服务端、凭据保护、JSON）。底层依赖 EUI-NEO GUI 框架。

关键依赖铁律：core 不得依赖 platform（仅允许一种有限的例外——头文件中声明但不包含实现的类型桥接，例如 eui::Color 值类型和组件主题的纯映射函数，它们不引入任何平台层的编译链接依赖）；module_* 之间不得互相依赖（工具调用走 ToolRegistry 按名间接触达，事件通信走 EventBus）；platform 可依赖 core。

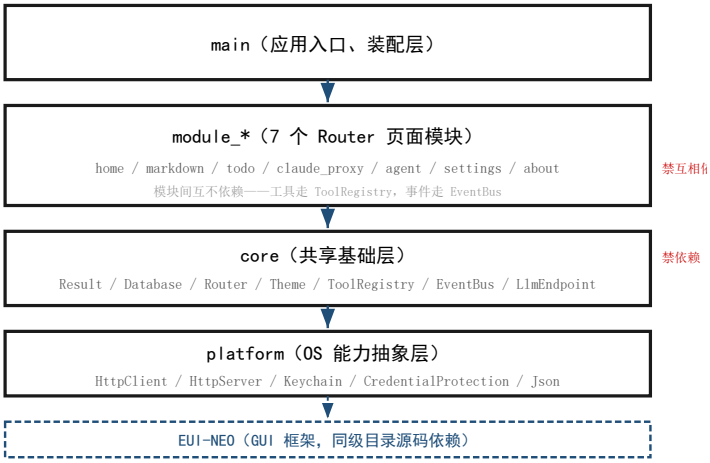


图 2-1 DevDesk 四层架构分层框图。实线箭头为允许的依赖方向，虚线为源码级引用（不进入链接）。红色标注为架构铁律。

2.2 执行循环机制

智能体的核心执行器是 AgentLoop（单次任务运行循环，流程见第三章 3.1 节的五阶段模型及与 DevDesk 实现的逐段对照）。其工作过程可分四步理解：

第一步，启动与组装。从 Scheduler 获得启动信号后，AgentLoop 将三块信息组装成一次 LLM 请求——全局运行环境提示词（由 Harness 内置生成）、本条任务自己填的角色设定、以及任务目标本身。

第二步，调用与解析。将组装好的提示词发给 LLM，等待响应。如果响应是纯文本回答，运行即告结束；如果响应中含有工具调用指令（tool_use），则进入工具执行通道。

第三步，工具执行。按工具名查找 ToolRegistry，检查权限（当前为全局破坏性操作开关，允许则执行、拒绝则终止），同步执行工具，将结果回喂到消息历史中。每轮执行前检查三个护栏——迭代次数是否超上限、运行时长是否超预算、Token 累计是否超阈值——任一超限即标记 limited 并终止运行。

第四步，迭代与落库。将本轮的用户提示词、模型响应、工具调用结果全部写入 agent_messages 审计表，然后回到第二步继续下一轮。运行结束时（completed / failed / cancelled / limited），把汇总信息写入 agent_runs 表。整个过程中，run 的状态变化通过 EventBus 推送给 UI，详情弹窗实时刷新。

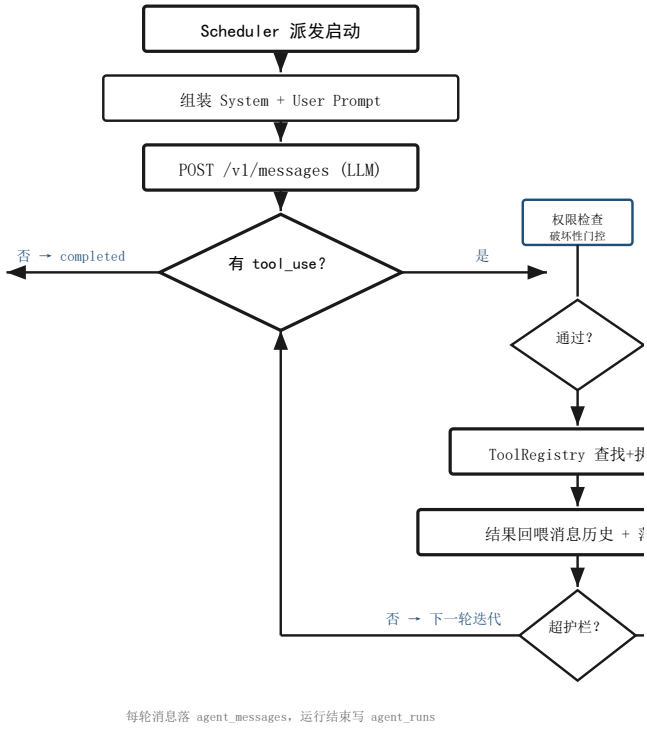


图 2-2 AgentLoop 执行循环流程图。方框为过程步骤，菱形为判断节点，实线箭头为控制流。

2.3 触发调度体系

智能体支持六种触发类型，由 Scheduler 统一管理：手动（UI 按钮）、定时 cron（按 cron 表达式周期调度，如"工作日每天 9 点"）、每天首次、闹钟一次性、事件（EventBus 驱动，首批三个动作事件——待办新建/完成/笔记保存）、常驻（手动入队）。共用闸包括同任务不并发、事件冷却（默认 300 秒，同任务不同事件互不压制）、运行次数上限达上限自动停用。

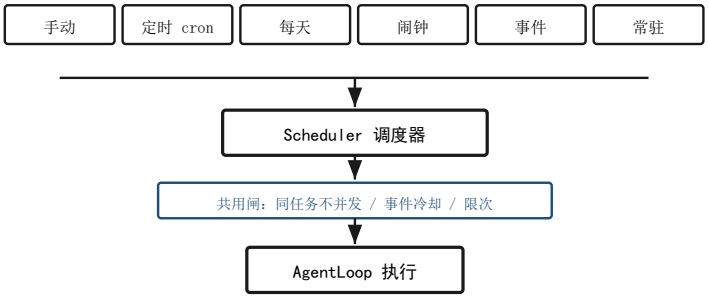


图 2-3 触发调度示意图。六种触发源汇入 Scheduler，经共用闸校验后派发 AgentLoop 执行。

2.4 护栏与审计

表 2-1 预算护栏体系现状

护栏	实现位置	说明
迭代上限	AgentTask::maxIterations（默认 10）	超限后结束运行，标记 limited
墙钟上限	AgentLoop 内 wallClockBudgetSec	min（迭代数 x 60s，30min）
Token 上限	Anthropic usage 字段累计	200K token 硬上限

工具结果截断	AgentLoop 32KB 硬上限	按 UTF-8 边界截断加 [truncated] 标记后回喂
单一运行闸	startRunExclusive	同任务同时只允许一个 run

审计体系由三张数据库表构成：agent_tasks（任务定义）、agent_runs（运行实例，含状态 / 耗时 / Token / 错误）、agent_messages（消息审计，含角色 / 内容 JSON / 工具调用 ID / 时间戳）。三表级联存储，支持运行记录逐轮回放。

2.5 工具与扩展点

表 2-2 智能体工具清单（按组）

组	工具	说明	破坏性
应用	app.stats / app.notify / app.now	统计数字、系统通知、当前时间	否
笔记	md.create / md.list_tags / md.delete	创建笔记、列标签、删除笔记	md.delete 是
待办	todo.create / todo.move / todo.complete / todo.list	创建、移动、完成、列待办	否
Claude	claude.list_profiles / claude.get_active / claude.switch_profile / claude.test_connection / claude.recent_logs	Profile 管理、切换、测试、日志	切换是
网络	net.fetch / hot.list	HTTP GET 抓取、四源热搜降级	否

扩展一个工具的标准步骤（已按现有模块接入路径设计、尚未有外部第三方工具按此流程完整接入验证）：在所属模块实现 core::ITool（名称 / Schema JSON / execute / isDestructive）→ 在模块 onStart 里 ToolRegistry::registerTool → 在 onStop 里 unregisterTool。模块隔离守卫自动检查 module_agent 与工具方互不 include。扩展一个事件只需两步：EventBus 加常量 → 发布方调 emit → 订阅方 subscribe。

2.6 模块依赖反转

四个核心抽象实现了模块间依赖反转（即高层模块不直接依赖低层模块、两者都依赖抽象接口——此处指 agent 模块通过 ToolRegistry 等 core 层抽象按名查找工具，而非直接 include 业务模块头文件），确保智能体模块与各业务模块互不依赖：ToolRegistry（agent 按名查找工具）、LlmEndpoint（agent loop 不依赖代理或具体传输层）、EventBus（发布方与订阅方只 include core 头）、LlmProfileSource（agent 从 claude_proxy 获取 Profile，互不依赖）。

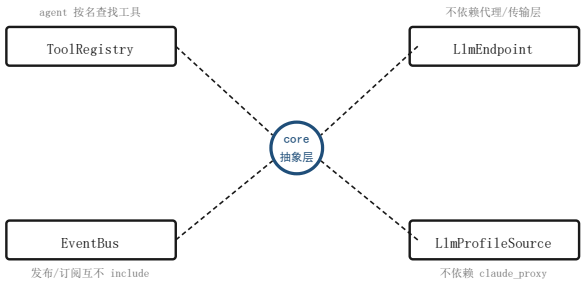


图 2-4 模块依赖反转关系图。四个抽象接口定义在 core 层，agent 通过接口间接触达各业务模块，互不产生编译期依赖。

第 三章 运行原理

> **本章要点：**讲解智能体三项核心机制的工作原理——Harness 五阶段循环、技能（skill）的渐进披露与上下文注入、MCP 协议的调用路径。最后从用户视角梳理扩展新能力的完整流程（业务面 + 技术面）。素材来自 12 号报告（AI Agent Handbook）、07 号报告（OpenCode 源码吸收）、06/11 号报告，以及本地克隆的 anthropics/skills 规范和 opencode2 源码。

3.1 Harness 基本原理与工作流程

智能体应用的核心公式是 Agent = Model + Harness [11]。模型提供认知能力（理解任务、推理方案、生成输出），但它不知道自己运行在什么环境里、有哪些工具可用、每次能花多少 Token。Harness 的职责就是把模型缺少的这些"外部约束和上下文"补上，让模型在一个有边界、可审计、可终止的环境中执行任务。

手册（12 号报告第 4 章）将 Harness 的执行内核抽象为一个五阶段循环 [11]——Prepare（准备）→ Model（调模型）→ Act（执行）→ Observe（观察）→ Verify（验证）。这个循环与我们 DevDesk 目前已实现的 AgentLoop 在结构上同构，各阶段职责如下：

Prepare（准备）。Harness 侧：组装本次调用的上下文——从 Context Builder 管线中收集环境信息、任务状态、技能指令、工具清单，按优先级和 Token 预算分配到系统提示词的各层（详见 3.2 节的六层编译表）。DevDesk 对应：AgentLoop 在每轮开始前从 AgentStore 读取任务定义（角色设定 + 任务记忆 + 触发配置），加上全局运行环境提示词，组装成完整的 messages 数组。

Model（调模型）。Harness 侧：将组装好的提示词发给 LLM，附上本次允许的工具定义列表。模型返回文本回答或工具调用请求。DevDesk 对应：通过 LlmEndpoint 抽象，经 DirectLlmClient 直连上游 API，POST /v1/messages，返回内容 stream 解析。

Act（执行）。Harness 侧：解析模型返回的工具调用——校验参数 Schema → 身份绑定 → 策略判定 → 审批（如果需要）→ 执行 → 捕获输出。DevDesk 对应：ToolRegistry::find(name) 查找 → 检查 allowDestructive 开关 → 同步执行 execute() → 32KB 截断保护。

Observe（观察）。Harness 侧：将工具执行结果结构化后回喂到对话历史中，同时写入 Event Log（只追加的不可变记录）和 Context Manifest（本轮模型到底看到了什么）。DevDesk 对应：工具结果追加进 messages 数组，本轮完整记录写入 agent_messages 表。

Verify（验证）。Harness 侧：检查停止条件——是否达到迭代上限、墙钟预算是否耗尽、Token 是否超阈值、Plan 是否有进展。满足任一条件则终止循环并产出 outcome（完成/失败/中断）。DevDesk 对应：每轮结束后检查三个护栏（maxIterations / wallClockBudgetSec / token limit），撞闸即标记 limited 或 failed。

手册同时归纳了 Harness 编排层需要提供的七大核心能力 [11]：循环与终止控制、指令工程（Prompt 分层编译）、规划与委派、工具管理、中间件与策略钩子、上下文压缩与续行、人在回路的验证点。DevDesk 当前已覆盖其中四项（循环控制、工具管理、策略钩子即权限闸、人在回路即破坏性操作审批），规划委派（对应 D/E 期角色分工）、压缩续行（对应长对话截断策略）、指令工程精细化（对应 A 期角色设定栏）为后续增量。

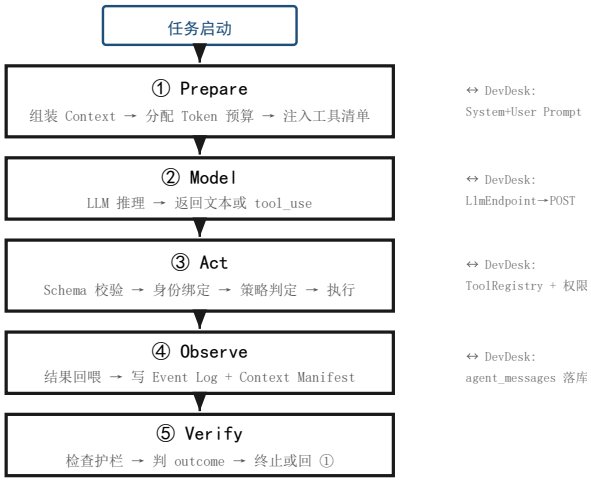


图 3-1 Harness 五阶段执行循环。右侧标注 DevDesk AgentLoop 的对应实现，两者在结构上同构。

3.2 技能的调用与实现

技能（skill）是一份教模型怎么干某类活的说明书，由 Anthropic 在 anthropics/skills 仓库 [30] 中定义了标准格式——一个 SKILL.md 文件，包含 YAML 前置元数据（frontmatter：技能名、触发描述）和 Markdown 正文（给模型的详细指令）。社区技能活跃——awesome-claude-code（56k stars）、awesome-claude-code-subagents（161 个子 agent）[29]，均为同格式的纯文本指令文件。

表 3-1 系统上下文六层编译结构

层序	层名	内容	来源	典型 Token 量级
1	Platform Policy	平台级安全与使用策略，全平台统一	Harness 内置	~200-500
2	Agent Contract	本 Agent 的身份契约——为谁工作、接受什么、交付什么、禁止什么	任务定义	~100-300
3	Tenant / Project Rule	租户或项目级规则与偏好（如特定代码规范）	项目配置	~50-200
4	Runtime Reminder	运行时的动态提醒——当前时间、OS 类型、工作区路径、git 分支、预算余量	Harness 环境注入	~200-400
5	Selected Skill	本次任务匹配到的技能指令（经渐进披露，只注入匹配到的那几条）	技能目录扫描	每技能 ~300-1500
6	Tool Descriptors	模型可调用的工具清单（名称 + 描述 + JSON Schema）	ToolRegistry 导出	~100-500/工具

技能采用渐进式披露（progressive disclosure）机制 [11]，分三层控制注入量，避免一次性把全部技能塞进上下文：

发现层。Harness 只在系统提示词中列出每个技能的名字和一句话适用场景（每条约 30-50 Token），模型据此判断"这个任务可能需要哪个技能"。这是最省 Token 的一层——20 个技能只需约 1000 Token。

选择层。当模型识别到某个技能匹配当前任务时，Harness 加载该技能的 Manifest（技能清单，约 200-500 Token），含工具声明、事件订阅、输入输出契约。模型据此确认是否真需要这个技能。

执行层。模型确认需要后，Harness 按需读取技能的完整 SKILL.md 正文（约 300-1500 Token），注入当前轮次的系统提示词第 5 层。执行完毕后即从上下文中移除，不在后续轮次中滞留。

三层机制的整体效果是：全量 20 个技能完整注入需约 20000 Token，而渐进披露的实际开销通常在 2000-4000 Token——节省约 80% 的上下文占用。DevDesk 的 C 期技能导入器以 anthropics/skills 的 SKILL.md 格式为起点，工具注册参照 OpenCode 的四元组（名称 + 描述 + JSON Schema 参数定义 + 执行函数）[10]，外部技能带着自己的参数定义进来、不需要提前编译。

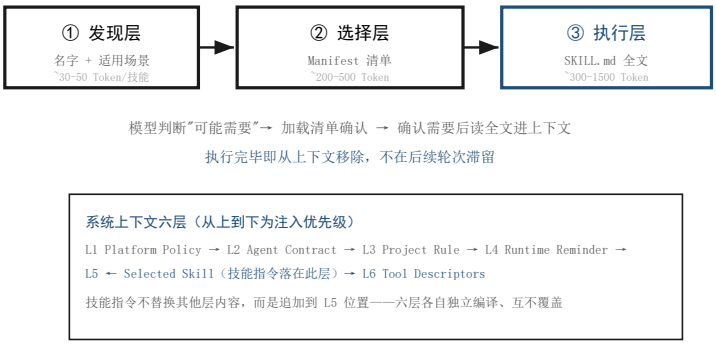


图 3-2 技能渐进披露三层流程与系统上下文六层编译结构。模型从发现层按需递进到执行层，避免一次性全量注入。

3.3 MCP 的调用与实现

MCP（Model Context Protocol，模型上下文协议）是一种客户端-服务器协议，让 AI 应用以标准方式连接外部工具和数据源 [7]。协议采用 JSON-RPC 2.0 作为消息格式，客户端（如 DevDesk）发起连接后，通过 tools/list 方法发现服务器上注册的工具清单（名称 + 描述 + JSON Schema 参数定义），后续模型发起的每次工具调用都经 JSON-RPC 封包发送到服务器执行，结果原路返回。

一次完整的 MCP 调用流程：客户端连接到 MCP 服务器（支持 stdio 进程通信或 HTTP/SSE 传输）→ 调用 tools/list 获取工具目录 → 将工具 Schema 合并进 ToolRegistry 统一注册表 → 模型发起 tool_use → Harness 判断该工具标记为 protocol="mcp" → 不走本地 execute()，转为 JSON-RPC tools/call 请求发送到 MCP 服务器 → 服务器返回结构化结果 → Harness 回喂模型 [10][38]。

MCP 与自建工具注册表的关系不是替代，而是分层。自建工具注册表是内核的"单一真相源"——所有工具（无论来自哪个模块、哪种协议）都通过同一个 ToolRegistry 注册，权限闸、审计、预算护栏在此统一执行。MCP 协议适配层是对外的一层"口音转换"——当工具标记为 protocol="mcp" 时，执行路径从本地 C++ 函数切换为 JSON-RPC 远程调用 [38]。这种分层设计（ADR 0004）保证了：(1) 内核不依赖任何外部协议，单一注册源的管理成本恒定；(2) MCP 标准化浪潮到来时，只需扩展 ProtocolAdapter、不动内核。

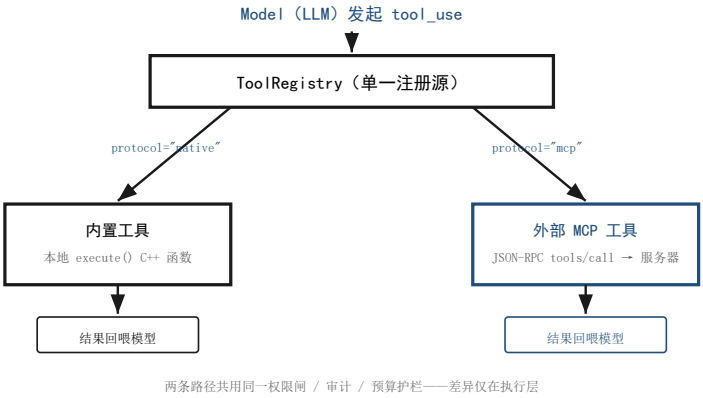


图 3-3 内置工具与 MCP 双路径对比。ToolRegistry 为单一注册源，protocol 字段决定走本地执行还是 JSON-RPC 远程调用。权限闸、审计、预算护栏在两条路径上统一执行。

3.4 用户如何扩展能力

对用户来说，给 DevDesk 智能体加一个新能力，对标 Claude Code 有三种安装面 [1][30]：插件市场（/plugin marketplace add，一键发现并安装）、技能目录（把 SKILL.md 文件放到指定目录下）、MCP 配置文件（在 .mcp.json 里声明服务器地址和参数）。DevDesk 的规划中，C 期技能导入器覆盖了"技能目录"这一面——用户把 SKILL.md 文件放入技能目录即可被自动发现和注册。MCP 客户端通道（评估项）覆盖"MCP 配置文件"面。插件市场对应社区生态，暂不在路线图中。

表 3-2 扩展能力安装面对比（Claude Code vs DevDesk）

安装面	Claude Code 方式	DevDesk 对应机制	差异
插件市场	/plugin marketplace add，一键安装	暂未规划（依赖社区生态）	DevDesk 无在线市场，技能靠手动获取
技能目录	将 SKILL.md 放入项目 .claude/skills/ 或用户级 ~/.claude/skills/	C 期导入器：将 SKILL.md 放入指定目录 → 自动扫描 → 注册进 ToolRegistry	机制相同；DevDesk 以 anthropics/skills 的 SKILL.md 格式为起点
MCP 配置文件	.mcp.json 声明 MCP 服务器地址、参数，Claude 启动时连接并发现工具	C 期评估 MCP 客户端通道：用户提供服务器地址 → ProtocolAdapter 连接 → 工具自动注册进 ToolRegistry	DevDesk 的内核为自建 ToolRegistry，MCP 工具经适配层接入而非直接暴露

用户扩展能力的技术层面遵循一条统一的注册管线：目录扫描（Harness 启动时扫描技能目录和 MCP 配置）→ 清单解析（读取 SKILL.md 的 frontmatter 和正文，或 MCP 服务器的 tools/list 响应）→ 注册进 ToolRegistry（单一真相源，所有工具在此汇聚）→ 权限初始化（默认最小权限：破坏性工具默认禁用，只读工具默认放行）→ 界面呈现（工具列表、技能清单在智能体页面的"可用工具"区可见、可逐条停用或卸载）。卸载即注销——从 ToolRegistry 移除对应条目，界面同步更新，不留残留注册项。整个生命周期中，权限闸、审计、预算护栏对所有工具（无论来源）统一执行。

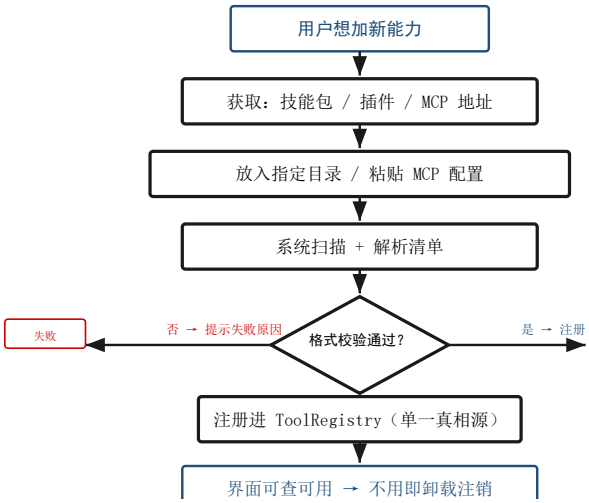


图 3-4 用户扩展能力业务流程图。从获取到注册到卸载，全生命周期受权限闸与审计统一管控。菱形为格式校验判断点，含失败分支。

第 四章 调研结论

> **本章要点：**重组七路调研结论，按主题（行业格局 / Harness 机制 / 工具生态 / GitLab / 可靠性与自主性 / 未来趋势）整理，每节先给结论与证据。开头为结论总表——12 条关键结论，每条附来源引用号及对路线图哪期哪条的意义。

#	结论	来源	对项目的意义
1	Claude Code 已验证 Harness 骨架可行，DevDesk 的"工具注册表 + 事件触发 + agent loop"架构方向正确。	[1] [2] [4]	验证 A 期收口基线；四层架构及依赖反转设计直接对齐该趋势
2	MCP 已是跨客户端事实标准，预计 2027-2028 年进入 IETF/W3C 标准化。自建工具注册表如不做 MCP 适配层，会在标准化浪潮中被边缘化。	[5] [6] [7]	A 期即埋 protocol 适配缝（约一天工作量）；C 期加 MCP 客户端通道评估
3	MR 留人审是行业共识——六大产品无一主张全自动合并。DevDesk 的"PRD→MR 流水线、MR 留人审"定位正确且务实。	[3] [8] [9]	确证 ADR 0001；E 期三层质量闸为行业标配
4	OpenCode v2.1.1 的权限三元组可直接搬到 C 期权限升级，相比现有全局开关是按工具粒度控制的成熟方案。	[10]	C 期权限逐工具升级的直接模板；07 报告 Top 1 吸收项
5	任务状态十态枚举 + 多维预算护栏 + 无进展检测是经企业级实证的任务执行模型。	[11]	A 期状态多维化、D1 无进展检测、E 期团队级停止条件的直接设计输入
6	开源 Agent 框架共同趋势：agent loop 极简核心 + 能力快插模式。DevDesk 的"轻资产原则"与此一致。	[12] [13] [14]	确证 ADR 0004；pydantic-ai Harness 模式为 C 期架构参考
7	GitLab MCP Server 官方实现已归档，无 MR 审查工具。社区搜索无可靠结果——B3 工具面需自建。	[15]	B3 自建五件套为最小承诺；E 期建 MR 四件套为追加
8	reviewdog 是 GitLab MR 自动审查的事实标准，其 diff-filter 模式可直接抄进 B3 设计。	[16]	B3 审查策略的直接参考——只分析 MR diff 变更行
9	学术界一致结论：自我评价必须结构化（rubric，即打分表），不能给模型自由打分空间；审查者与执行者必须分离。	[17] [18] [19]	D2 做完自检的设计基石——机械打分表 + 分离审查者 + 不退化验证门
10	失败分类是重试的前提：瞬时错误退避重试（1s→2s→4s，上限 3 次），逻辑错误进反思链，同类错误 3 次熔断。	[20] [21]	D1 失败重试的精确设计参数；防"盲重试"消耗 Token
11	短记忆正在被上下文替代；跨会话记忆仍是刚需，形态从"自维护"转向"协议化外部存储（MCP Memory）"。	[11] [38]	不自建记忆系统；C 期评估 MCP Memory Server 接入
12	桌面 Agent 的持久竞争优势在隐私与数据主权——个人数据正从"存在服务端"回流到"存在本地"。	[24] [25] [26]	确证桌面常驻定位；所有用户数据默认本地存储

4.1 行业格局

2026 年 AI Coding Agent 已从"代码补全"演进为"自主多步任务执行"。Claude Code 验证了"终端 agent + skills/plugins + 多表面切换"的 Harness 路线 [1]；GitHub Copilot 推出五级代理体系（agent mode / plan

mode / subagent / custom agent / cloud agent）[3]；Cursor 被 SpaceX 收购后提出"self-driving codebases"愿景 [4]。桌面端自动化产品（n8n 的 agent+workflow 双引擎、Raycast AI 的 Screen Awareness + Memory [27][28]）验证了"桌面 AI 操作系统"方向。代码审查自动化已从实验变为标配——GitHub Copilot code review 覆盖所有 PR、Factory.ai 逐 PR 审查 [9]——验证了 DevDesk GitLab 审查对接路线的紧迫性。DevDesk 选型"桌面壳 + agent loop + 工具注册"在形态上已正确；Copilot 的 plan mode 提示 agent loop 可内置"Plan First"策略。

4.2 成熟 Harness 机制

OpenCode v2.1.1 源码提供了可直接吸收的权限三元组规则引擎——每 Rule = {action, resource, effect}，effect 取 allow/deny/ask，支持 * 通配 [10]。其 Agent 角色体系（mode: primary/subagent/all + tool 白名单 + model 绑定 + steps 上限）直接映射到 D/E 期角色切分。手册（12 号报告）的 Harness 执行内核（Prepare→Model→Act→Observe→Verify）[11] 与 DevDesk AgentLoop 同构（详见第三章 3.1 节），但在任务状态（十态 vs 五态）和预算护栏（六维 vs 三维）上更完整。建议权限从全局开关升级为逐工具三态（C 期，抄 OpenCode Rule 结构）；任务状态从五态升级为十态（A 期）；审计加 Context Manifest（C/D 期）。

4.3 工具生态与技能

Claude Code 技能生态活跃——awesome-claude-code（56k stars）、awesome-claude-code-subagents（161 个子 agent）[29]。官方 anthropics/skills 提供 SKILL.md 格式 spec + 渐进披露加载机制 [30]。开源 Agent 框架盘点发现六个有价值的参考——smolagents（~1000 行核心）[12]、pydantic-ai（Harness 快插模式）[13]、OpenAI Agents SDK（handoff 交接）[14] 等。建议 D-R2 技能导入先搬 2-3 个纯指令型技能验证可行性；C 期技能导入器以 anthropics/skills spec 为格式起点；工具注册参照 OpenCode 四元组。

4.4 GitLab 对接

reviewdog 是 GitLab MR 自动审查的事实标准 [16]。GitLab MCP Server 官方实现已归档，仅有 9 个仓库管理工具 [15]。webhook 事件接入推荐 signing token（HMAC-SHA256，一种通过数字签名验证消息来源和完整性、防止伪造请求的安全机制）[34]。建议 B3 自建五件套（列 MR / MR 详情 / 读 diff / 读讨论 / 读流水线）为最小承诺；E 期建 MR 四件套为追加；事件接入采用 webhook + HMAC-SHA256 推到本地 HTTP 服务，不走 GitLab CI。

4.5 可靠性与自主性

学术文献对验收自评的一致结论：LLM 自评存在位置偏见、冗长偏见、自我增强偏见 [17]。解法三招：结构化评分表（rubric）[18]、审查者与执行者分离 [18]、不退化门 [19]。失败重试的核心洞察来自 ExpeL [20]：三连败同类错误 = 规则缺陷，单次失败 = 概率噪声。ANNEAL [21] 发现持久符号修复让复发率从 72-100% 降到 0%。自唤醒三边界来自 Generative Agents [35]：最多 3 次、随时可取消、醒来先自检。建议 D1 按瞬时/逻辑分类重试加无进展检测；D2 用机械打分表加审查者分离；D3 自唤醒最多 3 次 + 取消令牌；D 期校准为"可监控的半自主"。

4.6 未来趋势与押注

2027-2028 推演 [38]：上下文窗口持续扩大使短记忆被"直接塞进上下文"替代；MCP 将进入 IETF/W3C 标准化；AI 生成代码占比上升后，代码审查从质量把关变为意图对齐审查——检查 AI 是否理解了正确的事情，而非检查实现细节 Bug。OS 级 agent 会吃掉通用助手市场，但不会吃掉专业工具市场（DevDesk 是专业开发者工具中枢，与 OS agent 是上下游关系）。路线图方向正确，无"全期报废"风险。最大过时风险在协议层——C 期不做 MCP 适配则 2027-2028 年被迫返工。建议 A 期埋 MCP 适配缝、不自建记忆系统、不自建 PRD 解析器和代码审查引擎——DevDesk 的价值是编排和集成。

第 五章 与本项目计划的结合

> **本章要点：**按 A→E 分期路线逐期展开，每期写出设计参考来源、评估项与对冲；列出五项已拍板决策；七条设计原则；吸收映射总表；结论→计划逐项对接表。

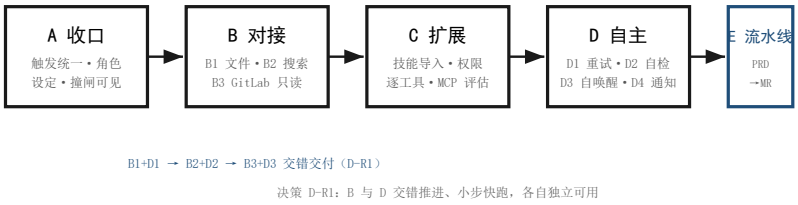


图 5-1 A→E 路线阶段图。B 期与 D 期按 D-R1 决策交错交付。

5.1 A 期：收口

A 期目标是把现有功能收到"顺手"——触发呈现统一、任务可定制、异常原因可见。八项内容：

1. **触发方式 UI 归四大类**（手动 / 按时间 / 等事件 / 一直跑）。底层保存方式不动，只改界面展示。

2. **任务加角色设定栏 + AI 草稿三件套**（目标 + 角色设定 + 记忆建议），权限与迭代上限等配置仍不听模型建议。

3. **撞闸和失败原因可见**——每次"没跑起来"或"跑了但失败"界面上都有可读原因文案。

4. **对冲——工具接口预留 MCP 协议适配缝**（每个工具定义加 protocol 字段，初始值 "native"，序列化走 ProtocolAdapter 抽象层）[38]。约一天工作量，防 C 期返工。

—— 以上四项为必做项，以下为增强项与候选 ——

5. **任务状态与预算护栏多维化** [11]：状态从五态升级为十态（CREATED / RUNNING / WAITING_INPUT / WAITING_APPROVAL / WAITING_EVENT / PAUSED / VERIFYING / COMPLETED / FAILED / CANCELLED）；预算护栏从三维升级为六维（步骤数 / 时长 / Token / 费用 / 工具调用次数 / 子任务并发）。"等审批"态依赖 C 期权限机制，C 期落地前该态不可达。

6. **运行记录补 outcome 字段** [10]：记录每次运行最终是圆满、部分完成还是中断。

7. **内置环境上下文注入（Claude Code 同款能力）**——每次运行组装六类环境事实进系统提示词：时空锚定（日期时间）/ 机器环境（OS 类型与架构、外网受限声明）/ 位置与仓库（工作区根路径、git 分支经 .git/HEAD 文件直读）/ 任务与运行元数据（任务名、运行编号、上次结果联动第 6 条）/ 能力与边界声明（可用工具总数、环境限制）/ 预算余量（剩余迭代/墙钟/Token，无人值守特有）。三条红线：只注环境事实绝不注密钥、按场景取最小充分集、环境变化检测补行不整块重注。

8. **候选——任务汇总一览提到列表层**：不打开详情弹窗即可横向比较各任务成功率与耗时，A 期方案时与列表行布局一并裁决。

设计参考来源：smolagents 的代码即行动格式 [12]、OpenCode 的环境上下文块 [10]（其 builtins.ts 中 core/date 和 core/environment 两个 Instruction 即是环境注入的标准范式——前者注入当前日期、后者注入工作目录/仓库类型/平台信息）、手册的任务状态与记忆结构化 [11]。最大风险是对冲项——MCP 适配缝如不在此期埋下，C 期返工代价是重构整个工具层。

5.2 B 期：对接能力

B 期分三批（B1 文件工具组+事件扩容 / B2 搜索工具+事件扩容 / B3 GitLab 只读五件套），每批配 D 期一块能力交错交付（D-R1）。B1 文件工具限专用工作区（ADR 0002），超出工作区路径一律拒绝。**B1 必答题**：智能体自己的工具写入（todo.*/md.* 等）要不要发事件——存在自触发循环风险（智能体写 → 触发事件 → 派发新运行 → 再写）；候选解法为事件携带来源标记、工具路径写入打标跳过。B1 立项时必须先拍死这道题再扩事件。B3 按五件套实现——列 MR / MR 详情 / 读 diff / 读讨论 / 读流水线状态 [15]，只读不产生外部可见影响。reviewdog 的 diff-filter 模式 [16] 直接抄进 B3。事件接入采用 webhook + HMAC-SHA256 签名验证推到本地 HTTP 服务 [34]，手动粘贴 MR 链接兜底。

5.3 C 期：扩展机制

C 期把"加新工具/事件"从改代码降为配置级操作。技能导入器以 anthropics/skills spec 为格式起点 [30]，注册参照 OpenCode 四元组 [10]。权限从全局开关升级为逐工具三态（ALLOW/DENY/ASK），抄 OpenCode 的 Rule 数据结构 [10]，高风险动作走 Preview-Approve-Commit-Verify 四步确认 [11]。MCP 客户端通道评估为开工必答题——内核保持自建工具注册表（单一真相源），对外加 MCP 协议兼容层 [38]。

5.4 D 期：自主性

D1 失败自动重试：瞬时错误指数退避（1s→2s→4s，上限 3 次），逻辑错误不重试而进反思链，同类错误 3 次按 10 分钟窗口 + 按任务独立计数熔断 [20][21]。重试前先做无进展检测 [11]。候选（旧账转入）：智能体直连路径的供应商故障自动切换——代理路径已有备用供应商单跳重试，直连未随迁；出现真实多供应商场景时随 D1 重试一并设计。D2 做完自检：结构化评分表（rubric，pass/fail 条目）、审查者与执行者分离（不同 prompt 甚至不同模型）[18]、不退化门 [19]。D3 自唤醒：复用闹钟底层机制，最多 3 次 + 取消令牌 + 醒来先自检 [35][19]。D4 失败升级通知。跨会话记忆评估对接 MCP Memory Server [38]。目标校准为"可监控的半自主"。

5.5 E 期：PRD→MR 流水线

E 期是智能中枢的目标形态：智能体承担 PM / QA / 开发三角色，从需求文档走到 GitLab 合并请求。流水线路径为需求文档输入 → PM 产出任务分解 → 开发产出代码变更 → QA 产出测试和审计结论 → 建 MR。全程内建三层质量闸（测试闸 / AI 审查闸 / 人工合并闸），MR 留人审合并（ADR 0001）。角色模型为每个角色 = 独立系统提示词 + 可用工具子集 + 阶段闸门 [10]。交接物为结构化 Handoff（目标/已知事实/待处理项/可用引用），下游必须引用上游交接卡片才能开工 [11][36]。QA 角色测试闸采用仿真三模式——回归测试用全替身（用模拟数据替代真实外部依赖，便宜可复现）、端到端用隔离真实依赖（在独立工作区里跑真实构建与测试）[11]。团队级停止条件 [11] 防流水线死循环。预算租约（Budget Lease）——子任务从父任务继承预算，父任务终止时子任务预算同时失效 [11]。



图 5-2 E 期 PRD→MR 流水线流程图。三个角色串行推进，每阶段产出经质量闸校验后进入下一阶段。三层质量闸缺一不可、顺序不换。

5.6 已拍板决策

表 5-1 五项已拍板决策

编号	决策	依据
D-R1	**推进节奏**：B 期对接与 D 期自主性交错交付、小步快跑。每批一块对接 + 一块自主，各自独立可用。	用户 2026-10-21 拍板
D-R2	**技能导入**：先搬 2-3 个纯指令型 Claude Code 技能，用技能说明书指导智能体验证可行性。优先序：code-review、brainstorming、skill-creator。	用户 2026-10-21 拍板
D-R3	**文件工具边界**：智能体文件工具只限专用工作区，业务数据必须走业务工具读写。系统目录和 .git 硬禁。	ADR 0002
D-R4	**GitLab 分档**：只读先做 → 写评论每条单独授权 → 建 MR 留到 E 期。	ADR 0003
D-R5	**流水线终点**：智能体产出 MR，合不合并由人拍板。质量闸内建在流水线里，最后一道门永远不自动。	ADR 0001

5.7 设计原则

综合七路调研，提炼七条贯穿 A→E 的设计原则。以下每条的编号和主题保留，但表述从单句格言展开为完整推理段落。

1. **工具注册表定位。** 内核维持自己的工具注册表作为单一注册源——所有工具在此汇聚，插件状态一处管理。对外通过 MCP 协议暴露，不在内核里同时维护两套工具定义体系。这避免了 dsh 的插件状态散落五处的反模式：激活/注销只需操作一份注册清单。

2. **MCP 适配时机。** 每个工具定义结构体现在就预留 protocol 字段（初始值 "native"），工具调用序列化与反序列化走独立的 ProtocolAdapter 抽象层。A 期做这件事约一天工作量——C 期再做，代价是整个工具层的重构。

3. **记忆系统选型。** 现有任务级便签维持现状不扩展。跨会话记忆直接对接 MCP Memory Server（官方维护的参考实现之一）。11 号报告的推演结论是：短记忆正在被上下文窗口的持续扩大所替代，自建记忆在 2027 年后没有存在理由 [38]。

4. **自我评价必须结构化。** 用 rubric（打分表）把验收标准拆成 pass/fail 机械验证项，不给模型 1-5 分的自由打分空间——这是 Reflexion、Self-Refine、LLM-as-a-Judge 三条文献的一致结论 [17][18][19]。审查者与执行者至少使用不同的 system prompt；预算允许时用不同模型路由。新策略提交前必须通过 hold-out 验证集（保留一部分不参与训练的数据），退化了就回滚。

5. **交接物是结构化 artifact。** 角色间的交接信息使用 JSON/Markdown 模板而非自然语言聊天记录，每条带版本号和溯源（谁改的、基于什么证据、什么时间）。下游必须在 prompt 中引用上游 artifact 的摘要才能开始工作——缺少摘要的，系统层直接拦截，不进入模型推理。这借鉴了 MetaGPT 的 SOP 编码（下游消费上游文档不能跳过）[36] 和 SWE-agent 的 ACI 概念（交接物应是机器可解析的结构化记录）[37]。

6. **自主性校准。** 每个自主决策保留可追溯的决策依据（trace）、可撤销的错误操作（回滚能力）、关键动作前的确认钩子（人审）。追求"完全无人值守"是 2028 年之后的事——当前所有产品离真正无人值守都还有距离，"可监控的半自主"比"假全自主"更保值 [38]。

7. **不自建已存在的组件。** PRD 解析器、代码审查引擎、技能市场这三个领域都已经有成熟的外部方案——PRD 解析有 Claude，代码审查有 code-review skill 和 MCP Server，技能市场有 MCP World。DevDesk 的价值不在于

重新发明这些组件，而在于把它们编排成一条从需求到 MR 的可审计流水线 [38]。

5.8 吸收映射

表 5-2 吸收映射总表

来源	机制/设计点	状态	对哪期
07	权限 Rule 三元组 + evaluate	直接吸收	C 期
07	工具注册四元组	直接吸收	B 期
07	Agent mode 三级 + tool 白名单	直接吸收	D/E 期
07	Session outcome 字段 + fork 机制	直接吸收	A/D 期
07	tool 执行进度回调	候选评估	C 期
12	任务状态十态 + 六维预算护栏	直接吸收	A 期
12	Context Manifest	直接吸收	C/D 期
12	权限三态 + Preview-Approve-Commit-Verify	直接吸收	C 期
12	完成验证五层强度分级	直接吸收	D 期
12	Handoff 交接四类信息	直接吸收	E 期
12	无进展检测 + Budget Lease + 团队停止条件	直接吸收	D/E 期
12	Agent Simulation 仿真三模式	直接吸收	E 期
07	ACP 协议全栈实现	不适用	—
07	Effect-TS Schema 类型系统	不适用（C++ 无等价物）	—
12	MCP 全协议与 MCP Gateway	不适用（明确不做）	—
12	AI 网关三层	不适用（桌面单用户）	—

5.9 结论→计划对照

表 5-3 调研结论与路线图计划逐项对接

#	结论摘要	对应路线图条目	对接动作
1	Harness 骨架方向正确	A 期（全部内容）	确证基线，无需调整
2	MCP 标准化趋势	A 期对冲 #4 + C 期 #4	A 期埋 protocol 适配缝；C 期评估 MCP 客户端
3	MR 留人审是行业共识	E 期质量闸 + ADR 0001	确证设计判断；三层闸不缩水
4	OpenCode 权限三元组可搬	C 期 #3（权限升级）	直接抄 Rule 结构 + evaluate 算法
5	任务状态十态 + 多维预算	A 期 #5（任务状态多维化）	按手册模型升级现有五态+三维护栏
6	开源框架轻核心+快插模式	C 期 #2（技能导入器）	确证轻资产原则；pydantic-ai Harness 为架构参考
7	GitLab MCP 官方已归档	B3 期（自建工具面）	五件套自建；评估不再等社区方案
8	reviewdog diff-filter 可抄	B3 期（审查策略）	只分析 MR diff 变更行

9	自我评价必须结构化	D2 期（做完自检）	机械打分表 + 审查者分离 + 不退化门
10	失败分类+熔断+无进展检测	D1 期（失败重试）	瞬时退避/逻辑反思/同类熔断三轨制
11	短记忆贬值、长记忆接 MCP	D 期评估项（跨会话记忆）	不扩展自建记忆；评估 MCP Memory Server
12	桌面隐私是持久竞争优势	设计原则 #7（数据主权）	默认本地存储、云端访问显式授权

表 5-4 B3 期工具面草案

工具	API 端点	参数	Scope
list_merge_requests	GET /projects/:id/merge_requests	project_id, state?, search?, page?, per_page?	read_api
get_merge_request	GET /projects/:id/merge_requests/:mr_iid	project_id, mr_iid	read_api
get_merge_request_changes	GET /projects/:id/merge_requests/:mr_iid/changes	project_id, mr_iid	read_api
get_merge_request_discussions	GET /projects/:id/merge_requests/:mr_iid/discussions	project_id, mr_iid, page?, per_page?	read_api
get_pipeline_status	GET /projects/:id/pipelines/:pipeline_id	project_id, pipeline_id 或 mr_iid	read_api

表 5-5 E 期建 MR 最小工具面草案

工具	API 端点	关键参数	Scope
create_merge_request	POST /projects/:id/merge_requests	project_id, title, description?, source_branch, target_branch, draft?	api
create_merge_request_note	POST /projects/:id/merge_requests/:mr_iid/notes	project_id, mr_iid, body, line?, line_type?, path?	api
create_merge_request_discussion	POST /projects/:id/merge_requests/:mr_iid/discussions	project_id, mr_iid, body, position?	api
merge_merge_request	PUT /projects/:id/merge_requests/:mr_iid/merge	project_id, mr_iid, merge_commit_message?	api（受人审约束）

第 六章 结论与建议

> **本章要点：**总判断——路线图方向正确，无"全期报废"致命风险；最大风险在协议层（MCP 适配）；三条近期建议。

6.1 总判断

基于七路调研的交叉验证，DevDesk 智能中枢路线图（A→E 五期）在方向上正确，不存在需要推翻重来的致命缺陷。**架构层**——四层分层 + 依赖反转设计（ToolRegistry / LlmEndpoint / EventBus / LlmProfileSource 四枢纽）与行业最佳实践一致。**路线层**——A 收口 → B 对接 → C 扩展 → D 自主性 → E 流水线的分期逻辑合理，B 与 D 交错交付兼顾"对外能力"和"对内智能"两条线。**决策层**——五项已拍板决策与 ADR 0001-0004 均经外部验证。

6.2 最大风险与对冲

最大风险在协议层：C 期如不做 MCP 适配，将在 2027-2028 年 MCP 标准化浪潮中被迫返工 [38]。对冲方案已在 A 期路线图中写入——每个工具定义预留 protocol 字段，序列化走独立 ProtocolAdapter 抽象层，评估约一天工作量。次要风险在记忆层：自建记忆系统在上下文窗口扩大和推理成本下降的趋势下面临贬值，建议不对现有自建记忆做扩展投入，转而评估对接 MCP Memory Server。

6.3 三条近期建议

第一，A 期开发埋 MCP 适配缝。在工具注册、AgentLoop 现有实现中，每个工具定义加 protocol 字段，序列化/反序列化走 ProtocolAdapter。这是成本最低的保值操作（约一天），防 C 期被迫重构整个工具层。同时开始验证代码即行动格式（smolagents 的 CodeAgent 模式）在 C++ agent loop 中的可行性。

第二，D 期不追求"完全自主"，校准为"可监控的半自主"。所有自主行为加 trace、回滚、人审钩子。D2 验收自评用机械打分表，审查者与执行者分离。D1 重试按瞬时/逻辑分类，加无进展检测。D3 自唤醒最多 3 次 + 取消令牌 + 醒来先自检。这些参数均有文献实证支撑 [18][19][20][21][35]，不是凭空设计。

第三，E 期各环节设计为可替换的外部能力接入点。代码生成可以是 DevDesk 自建 agent 也可以是 Claude Code via MCP；需求解析和代码审查不自建（Claude Code 已有 code-review skill、社区 MCP Server 已有成熟实现）。编排和集成才是 DevDesk 的价值。

附录 A 引用清单

以下引用均于 2026-10-21 抓取。编号与正文引用标号对应。

[1] Anthropic. Claude Code Overview. https://docs.anthropic.com/en/docs/claude-code/overview

[2] Anthropic. Agent SDK Overview. https://docs.anthropic.com/en/docs/agent-sdk

[3] GitHub. Copilot Features. https://github.com/features/copilot

[4] Cursor. Blog. https://www.cursor.com/blog

[5] Anthropic. MCP 官方公告. https://www.anthropic.com/news/model-context-protocol

[6] n8n. MCP Servers 集成文档. https://docs.n8n.io/build/integrate-ai/mcp-servers

[7] MCP. Architecture Overview. https://modelcontextprotocol.io/docs/concepts/architecture

[8] OpenHands. https://github.com/All-Hands-AI/OpenHands

[9] Factory.ai. https://www.factory.ai

[10] anomalyco/opencode v2.1.1. 本地克隆 D:\project\gui\opencode2. packages/core/src/permission.ts, packages/schema/src/agent.ts, packages/ai/src/tool.ts, packages/core/src/instructions/builtins.ts

[11] 阿里云. AI Agent Handbook. https://aliyun.github.io/ai-agent-handbook/contents.html. 第 2/4/5/6/11/13/16 章

[12] huggingface/smolagents. https://github.com/huggingface/smolagents

[13] pydantic/pydantic-ai. https://github.com/pydantic/pydantic-ai

[14] openai/openai-agents-python. https://github.com/openai/openai-agents-python

[15] modelcontextprotocol/servers-archived. GitLab MCP Server. https://github.com/modelcontextprotocol/servers-archived/tree/main/src/gitlab

[16] reviewdog/reviewdog. https://github.com/reviewdog/reviewdog

[17] Zheng et al. (2023). Judging LLM-as-a-Judge. arXiv:2306.05685. NeurIPS 2023

[18] Madaan et al. (2023). Self-Refine. arXiv:2303.17651

[19] Nguyen et al. (2026). RSEA. arXiv:2606.28374

[20] Zhao et al. (2023). ExpeL. arXiv:2308.10144. AAAI-24

[21] Hakim et al. (2026). ANNEAL. arXiv:2605.16309

[22] DevDesk 11 号报告 §1.1. 上下文窗口持续扩大的轨迹推演：Gemini 2M token + Claude Opus 5.5 成本下降 40%. docs/strategy/11-未来趋势与技术押注.md

[23] MCP Memory Server. modelcontextprotocol/servers 官方参考实现

[24] 联想天禧. PC 端系统级 Agent. 11 报告第 4 节

[25] Apple Intelligence. 本地处理优先. 11 报告第 4 节

[26] OpenOcta 八爪鱼. 信创友好桌面 Agent 框架. 11 报告第 4 节

[27] n8n. 构建与管理 Agent. https://docs.n8n.io/build/build-and-manage-agents

[28] Raycast AI. https://www.raycast.com/ai

[29] VoltAgent/awesome-claude-code-subagents. https://github.com/VoltAgent/awesome-claude-code-subagents

[30] anthropics/skills. https://github.com/anthropics/skills. 本地克隆 D:\project\gui\anthropic-skills. SKILL.md 格式含 YAML frontmatter + Markdown 正文指令

[31] obra/superpowers. code-review / brainstorming skills

[32] crewAIInc/crewAI. https://github.com/crewAIInc/crewAI

[33] browser-use/browser-use. https://github.com/browser-use/browser-use

[34] GitLab. Webhooks 文档. https://docs.gitlab.com/ee/user/project/integrations/webhooks.html

[35] Park et al. (2023). Generative Agents. arXiv:2304.03442

[36] Hong et al. (2023). MetaGPT. arXiv:2308.00352

[37] Yang et al. (2024). SWE-agent. arXiv:2405.15793. NeurIPS 2024

[38] DevDesk 11 号报告. 未来趋势与技术押注. docs/strategy/11-未来趋势与技术押注.md

[39] DevDesk ADR 0001. PRD→MR 流水线终点是 MR. docs/adr/0001-*.md

附录 B 决策记录卡

ADR 0001：PRD→MR 流水线的终点是 MR，合并决策保留给人

用户 2026-10-21 确定：智能体承担 PM / QA / 开发角色分工，产出 GitLab 合并请求；MR 之后的人工审查与合并不自动化。理由：MR 留人审是团队既有的质量闸，全自动合并的风险远大于省一道人审的收益。智能体的质量闸内建在流水线内部，但最后一道门永远是人。状态：已接受。

ADR 0002：智能体文件工具的读写边界——仅限专用工作区

用户 2026-10-21 拍板：文件读写工具只开放"数据根下的专用工作区目录"；笔记库、待办、配置库等业务数据一律经既有业务工具访问，禁止直写。理由：绕过业务工具直写会制造索引与运行态不一致。系统目录与 .git 硬禁。状态：已接受。

ADR 0003：GitLab 对接按外部可见性分档，回写视同破坏性

用户 2026-10-21 拍板：GitLab 工具组按外部可见性递增分三档——只读档首期即做；评论回写为第二档须任务级显式授权；建 MR 留到 E 期。理由：外部可见动作的错误不可撤回，风险语义与删除数据同类。状态：已接受。

ADR 0004：核心只留机制与接口，业务能力皆插件（轻资产原则）

2026-10-21 用户拍板。核心只保留"机制与接口"——智能体循环、工具注册、事件总线、触发调度、权限闸、审计、预算护栏。一切具体业务能力以可插拔形态接入，不进核心。接口质量硬判据 = 新能力以"注册 / 放文件"形态接入的步骤数（工具 4 步、事件 2 步）。理由：前人（Claude Code / opencode / pydantic-ai）已收敛于"极简循环 + 注册表 + 能力快插"形态，轻核心保证协议与能力都可以在标准化浪潮中低成本替换。状态：已接受。

附录 C 术语表

智能体（Agent）

DevDesk 内置的后台自动化执行者：接收任务目标，自主循环调用模型与工具完成它，全程留运行记录。

Harness（智能体运行时）

智能体循环之外的宿主机制统称：触发调度、工具注册、权限闸、预算护栏、运行审计、任务记忆。

AgentLoop

单次任务运行的执行循环：组装提示词 → 调 LLM → 解析结果 → 如有工具调用则执行工具 → 回喂结果 → 迭代，直至模型返回文本或撞护栏。

MCP（Model Context Protocol）

一种让 AI 工具连接外部数据和服务器的通用接口标准。由 Anthropic 开源，已成为跨客户端的工具连接事实标准。

PRD→MR 流水线

智能中枢的终极目标形态：从产品需求出发，智能体承担 PM、QA、开发的角色分工，产出合并请求交人审查。

MR（Merge Request）

GitLab 的代码变更提案，请求将一组代码变更合并进目标分支。

技能（Skill）

一份教 AI 怎么干某类活的说明书，格式为 SKILL.md（YAML 前置元数据 + Markdown 正文指令）。通过导入器注册进智能体运行环境。

专用工作区

数据根下为智能体文件工具开辟的唯一可读写目录；业务库只能经业务工具访问（ADR 0002）。

破坏性操作

不可逆副作用的工具动作（删除数据、切换配置、发布外部评论等）。任务未授权时一旦尝试，该次运行立即终止。

webhook

外部系统把事件"推"给我们的回调通道，用 HMAC-SHA256 签名验证来源和完整性。

Token

模型处理文字量的计数单位（按字/词折算），每次调用消耗分输入和输出两部分。

Budget Lease（预算租约）

子任务从父任务继承的预算许可：一次授权同时限定资源上限、操作范围、有效期与撤销方式。